

National University of Computer & Emerging Sciences

Karachi Campus

ASSIGNMENT 2

DATA STRUCTURE

Professor: Dr. Anum Qureshi

Section: BSE-3A

Student:

25k-3050 - Muhammad Zain

Github Repository:

[Assignment 2 Codes]

Question One

Re construct Binary tree

Given:

PreOrder: A, B, D, E, H, I, C, F, G

In Order : D, B, H, E, I, A, F, C, G

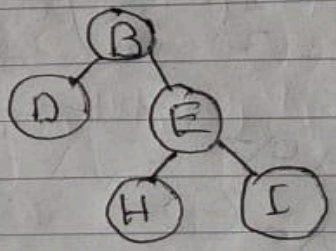
In InOrder the first element is root.

Root = A.

left subtree preorder: B D E H I

" " inorder : D B H E I

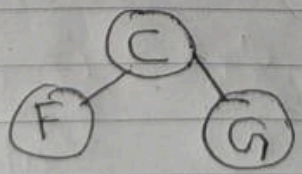
Post order : D H I E B



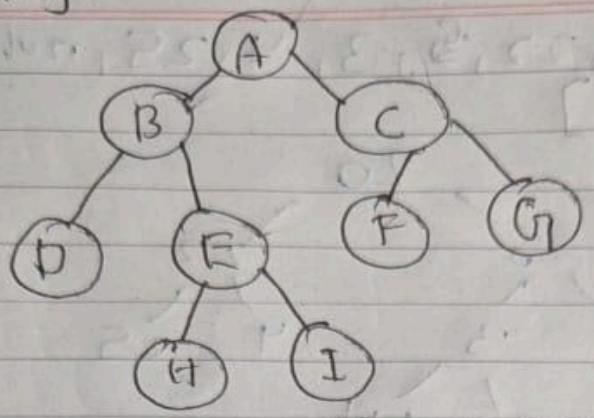
right subtree preorder: C F G

inorder: F C G

postorder :



Finally:



Postorder D H I E B F G C A

b)

n_0 : Leaf nodes

n_1 = nodes with 1 child

n_2 : ~~not~~ full node

tree with n nodes has n-1 edges.

$$n = n_0 + n_1 + n_2$$

Every node contributs its number of children as outgoing edges.

n_2 = 2 children n_1 = 1 so

$$n_1 + 2n_2 = n - 1$$

Put n

$$n_1 + 2n_2 = n_0 + n_1 + n_2 - 1$$

$$\boxed{n_0 = n_2 + 1}$$

Proved

The number of leaf nodes is always one greater than the number of nodes having two children. in every non empty bt.

Question 2:
Code:

```
#include <iostream>
using namespace std;
class node{

public: int data;
        node* left;
        node* right;
  public:
  node(int dt){
        left=nullptr;
        right=nullptr;
        data=dt;
}
};
node* createnode(int key){
        node* temp=new node(key);
        return temp;
        }
class BST{
public:
        node* root;
        BST(){
                root=nullptr;
        }
        node* insert(node* root, int key){
                if(root==nullptr){
                        root=createnode(key);
                        return root;
                }
                if(root->data>key){
                        root->left=insert(root->left, key);

                }else if(root->data<key){
                        root->right=insert(root->right,key);

                }
                return root;
        }
        node* search(node* root, int data){
                if(root==nullptr){
                        return root;
```

```
                }
                if(data==root->data) {
                return root;}
                if(data<root->data){
                        return search(root->left,data);
         }
                else{
                                return search(root->right, data);
                        }
}
node* findMin(node* root){
        if(root==nullptr){
                return root;
        }
        if(root->left==nullptr){
                return root;
        }
        return findMin(root->left);
}
node* findMax(node* root){
        if(root==nullptr){
                return root;
        }
        if(root->right==nullptr){
                return root;
        }
        return findMax(root->right);
}
void deleteKey(int key) {
   root = deleteNode(root, key);
}
node* deleteNode(node* root,int key){
  if(root==nullptr){
        return nullptr;
  }
  if(key<root->data){
        root->left=deleteNode(root->left, key);
  }else if(key>root->data){
        root->right=deleteNode(root->right, key);
  }
  else{
        if(root->left==nullptr && root->right==nullptr){
                delete root;
                return nullptr;
```

```
            }

   else if(root->left==nullptr){
          node* temp=root->right;
          delete root;
          return temp;
   }else if(root->right==nullptr){
          node* temp=root->left;
          delete root;
          return temp;
   }
   else{
          node* temp=findMin(root->left);
          root->data=temp->data;
          root->left=deleteNode(root->left,temp->data);
   }
   }
         return root;

}
node* inorderSuccessor(int key) {
    node* succ = nullptr;
    node* curr = root;

    while (curr != nullptr) {
        if (key < curr->data) {

            succ = curr;
            curr = curr->left;
        }
        else if (key > curr->data) {
            curr = curr->right;
        }
        else {
            if (curr->right != nullptr) {
                succ = findMin(curr->right);
            }
            break;
        }
    }
    return succ;
}
void display(node* root){
          if(root==nullptr){
```

```
                return;
        }
        display(root->left);
                cout<<root->data<<" ";
        display(root->right);
}
};
int main(){
        BST tree;
    tree.root = tree.insert(tree.root, 20);

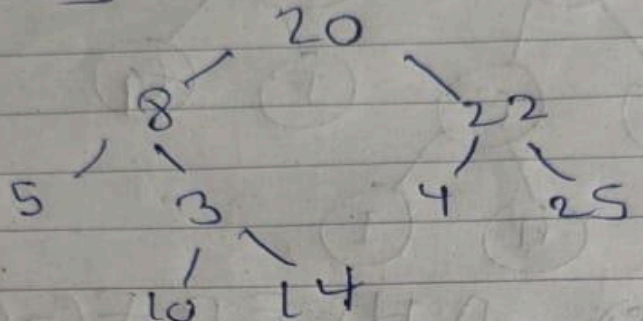
   tree.root= tree.insert(tree.root, 8);
   tree.root= tree.insert(tree.root, 22);
   tree.root= tree.insert(tree.root, 4);
    tree.root=tree.insert(tree.root, 12);
   tree.root= tree.insert(tree.root, 10);
   tree.root= tree.insert(tree.root, 14);

    node* target = tree.search(tree.root, 14);
    node* succ = tree.inorderSuccessor(14);
    tree.display(tree.root);
    cout<<"\n";
    tree.deleteKey(20);
    tree.display(tree.root);
        cout<<"The Successor is: "<<succ->data;

    return 0;
}
```

Q3 (a)

[20, 8, 22, 5, 3, 4, 25, null, null, 10, 14]



Pre order

20 = root

20, 8, 5, 3, 10, 14, 22, 4, 25

In Order

5, 8, 10, 3, 14, 20, 4, 22, 25

root = 20

Postorder

Root = 20

5, 10, 14, 3, 8, 4, 25, 22, 20

level order.

20, 8, 22, 5, 3, 4, 25, 10, 14

b)

i Height:

Height is the number of edges in the longest past from root to leaf.

20 - 8 - 3 - 10/14

or

3 edges = 3 height

Leaf Nodes

Leaf nodes are nodes with zero children.

5, 10, 14, 4, 25

Leaf nodes = 5

Nodes with two children

20, 8, 3, 22

Full Nodes = 4

Identity.

Leaves = Nodes with 2 children + 1

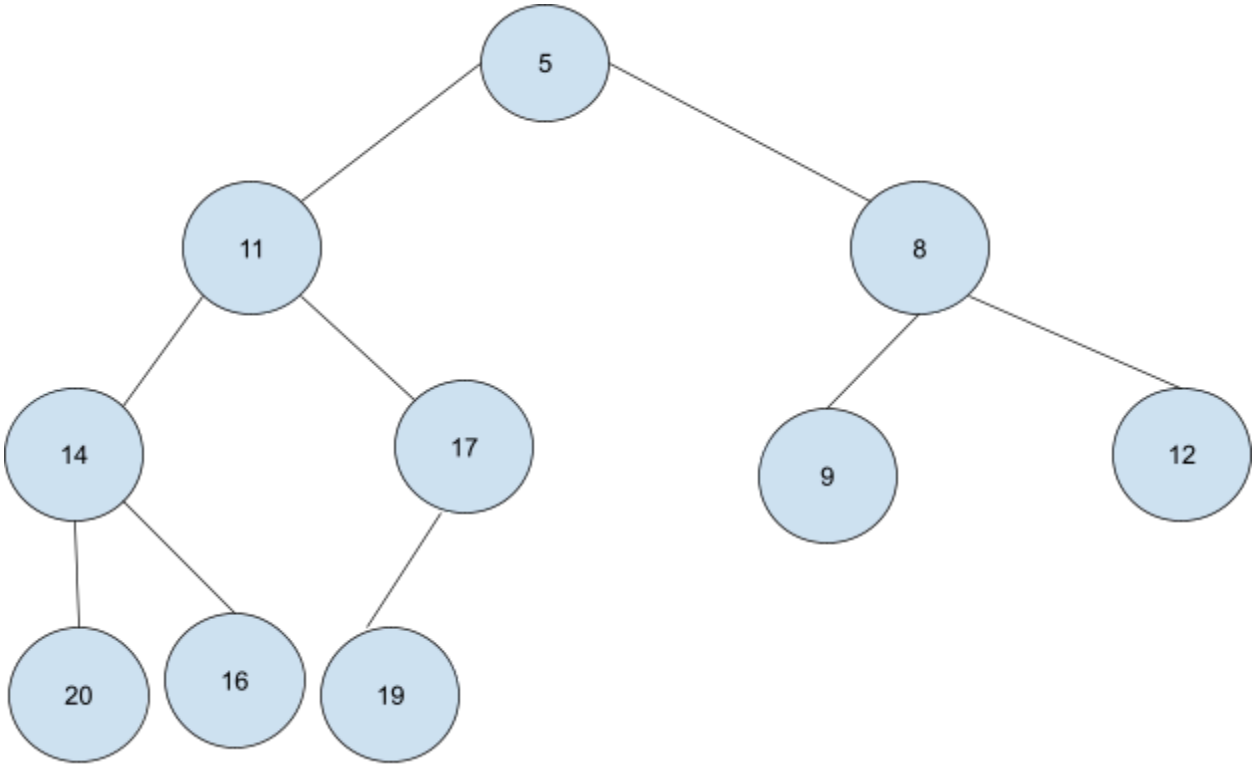
5 = 4 + 1

5 = 5

Question 4:

Given Array H:[5, 11, 8, 14, 17, 9, 12, 20, 16, 19]

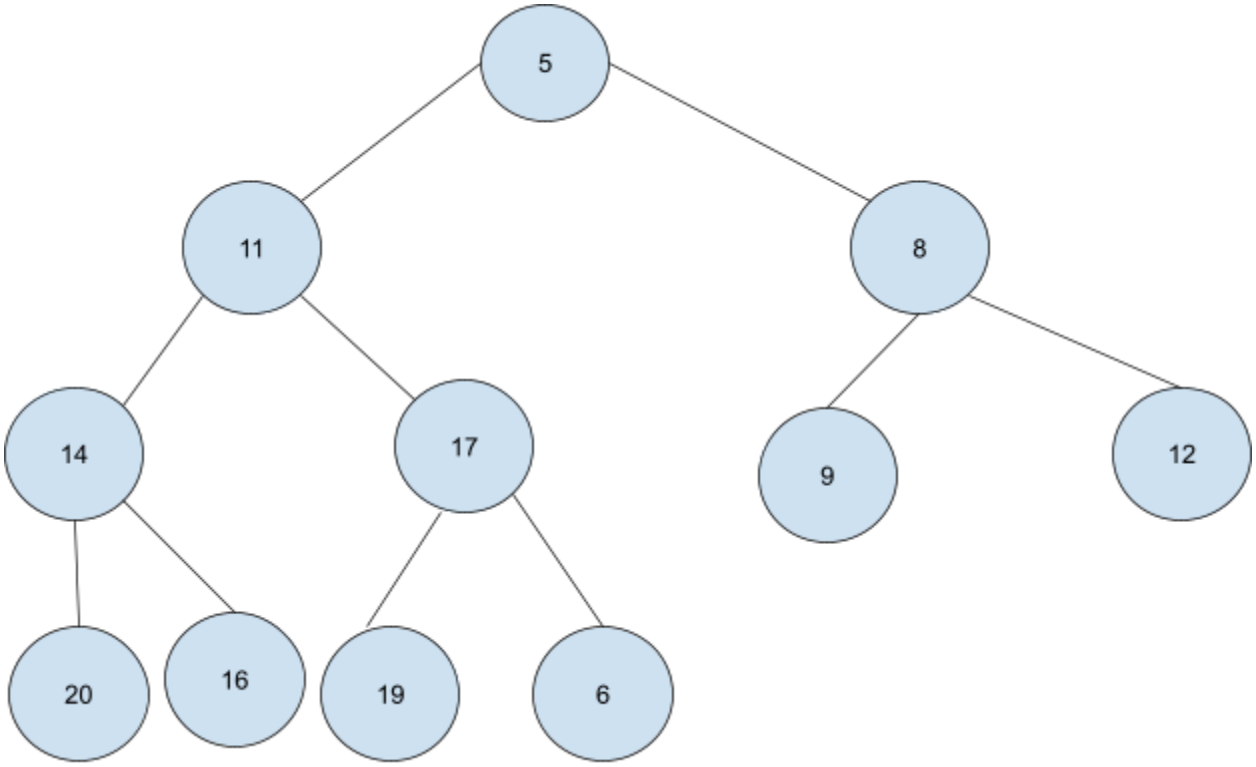
Binary Representation:



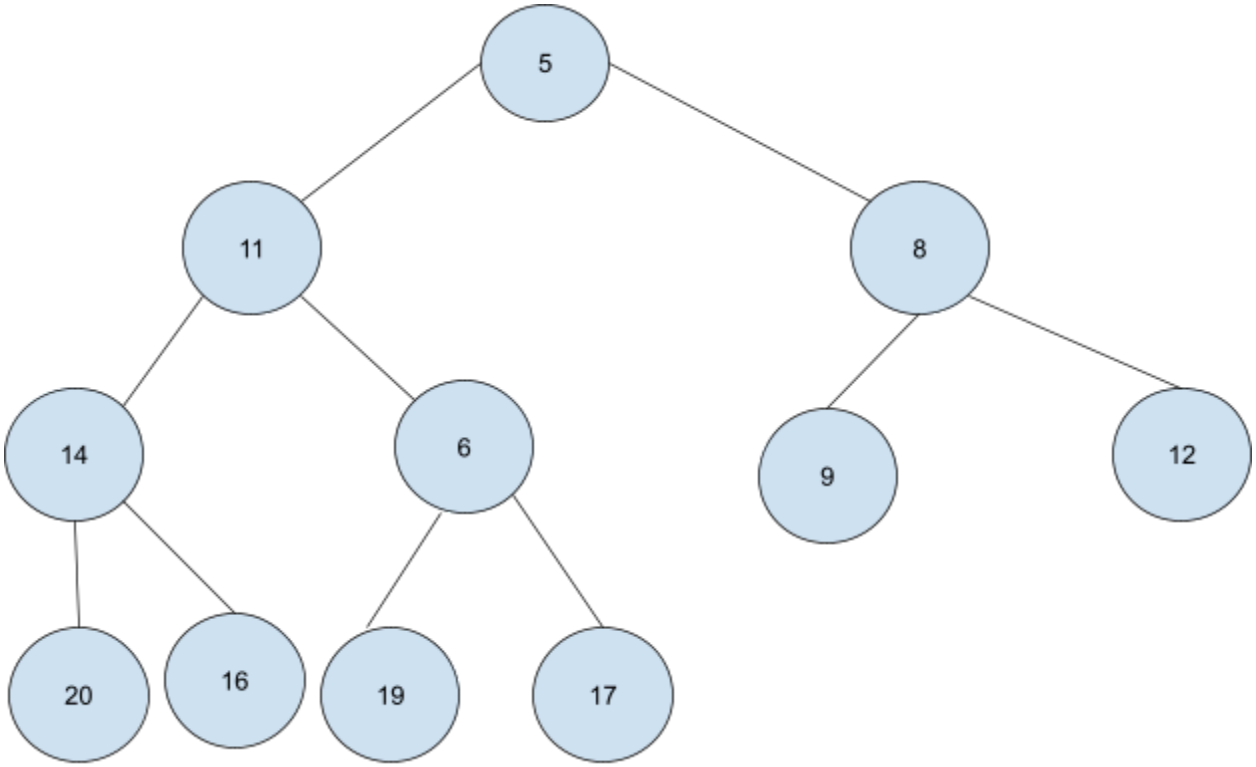
Parent	Children	Check
5	11, 8	True
11	14,17	True
14	20, 16	True
17	19	True
8	9,12	True

Inserting 6:

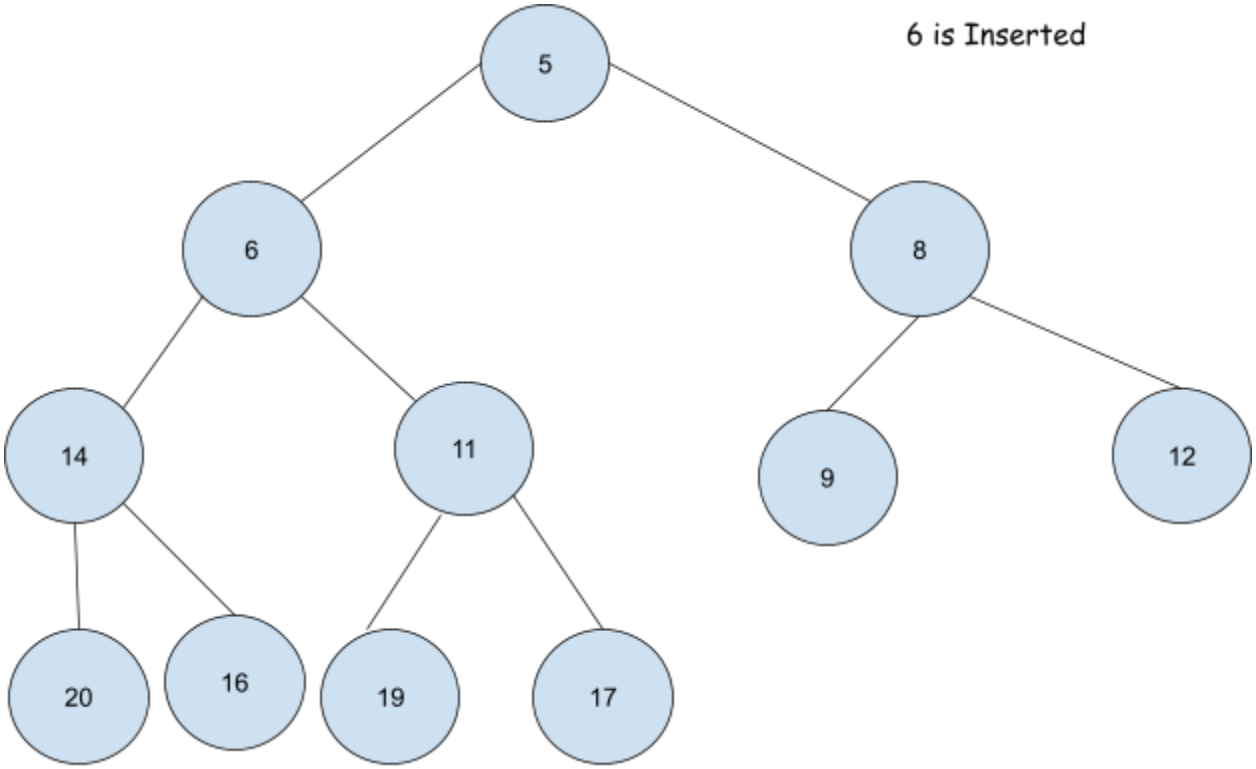
[5,11,8,14,17,9,12,20,16,19,6]



Comparing with its parent and swapping
[5, 11, 8, 14, 6, 9, 12, 20, 16, 19, 17]

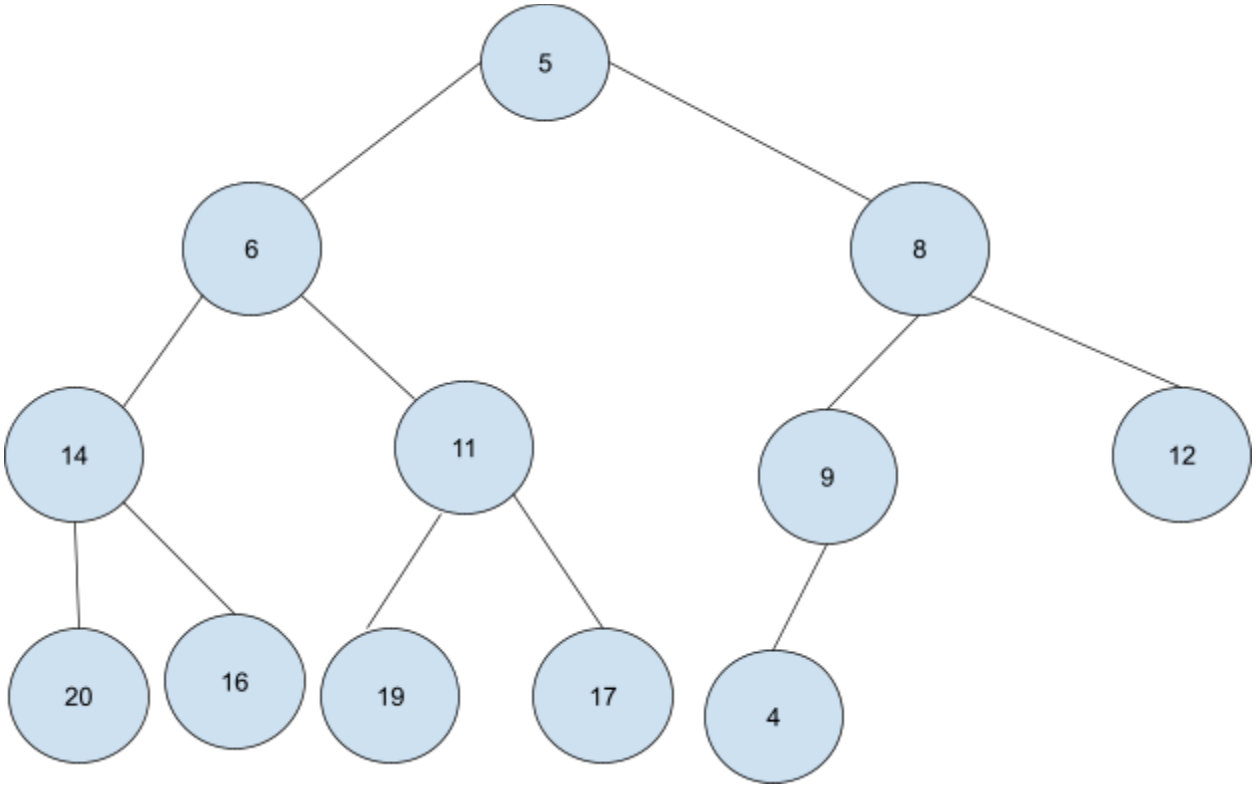


Comparing 6 with its Parent and Swapping
[5, 6, 8, 14, 11, 9, 12, 20, 16, 19, 17]
TotalSwaps=2

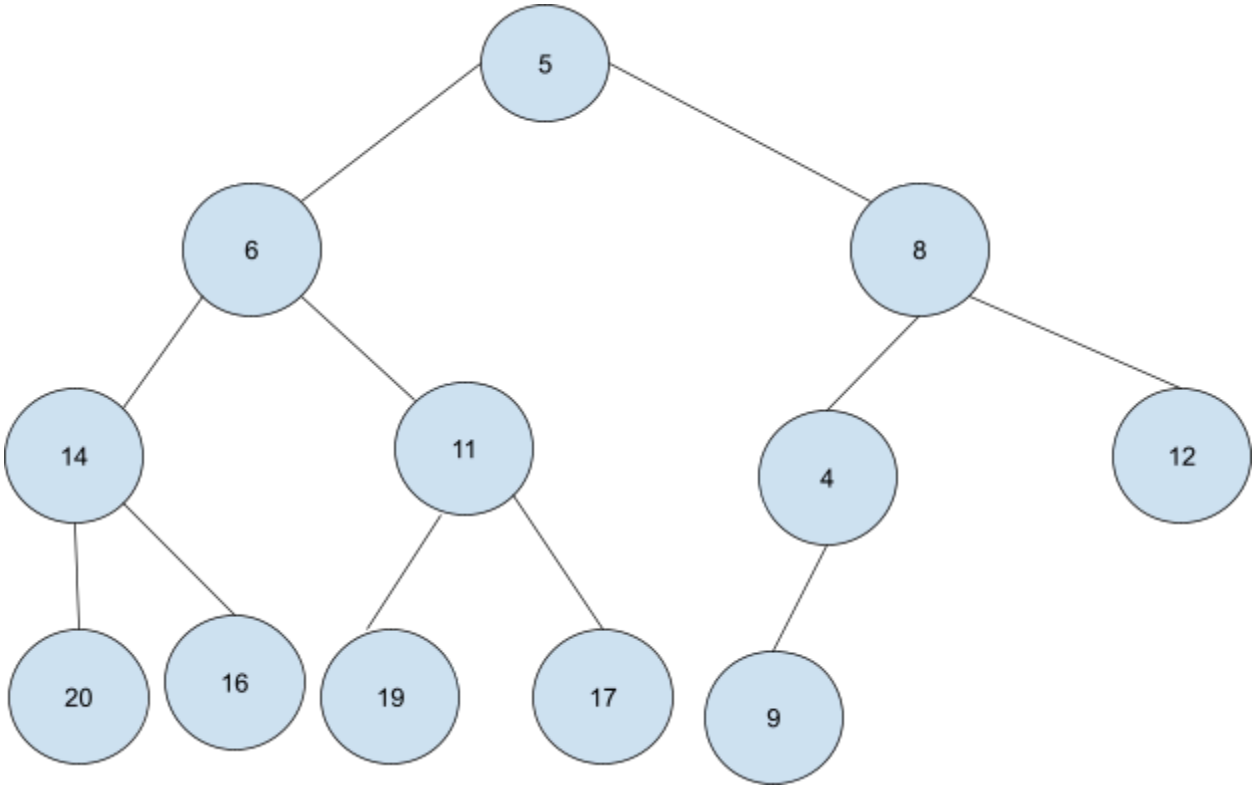


Inserting 4:

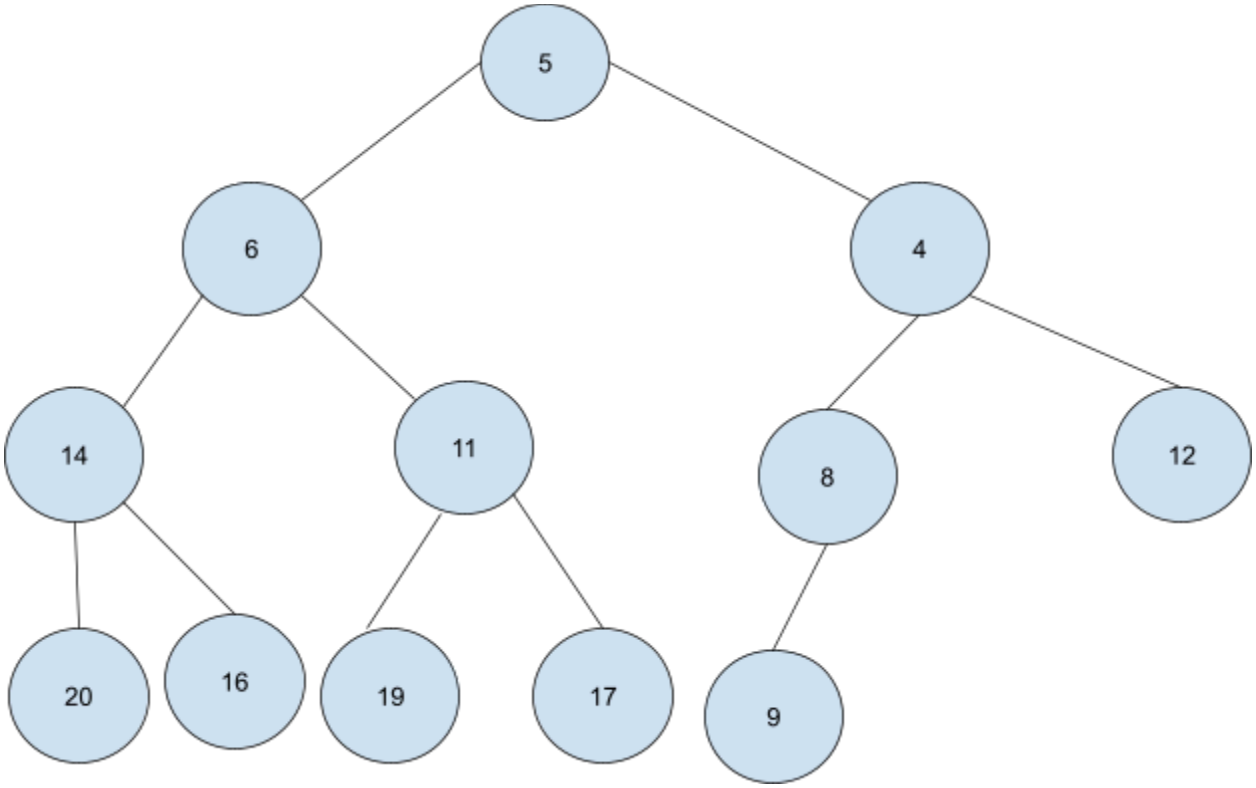
[5, 6, 8, 14, 11, 9, 12, 20, 16, 19, 17, 4]



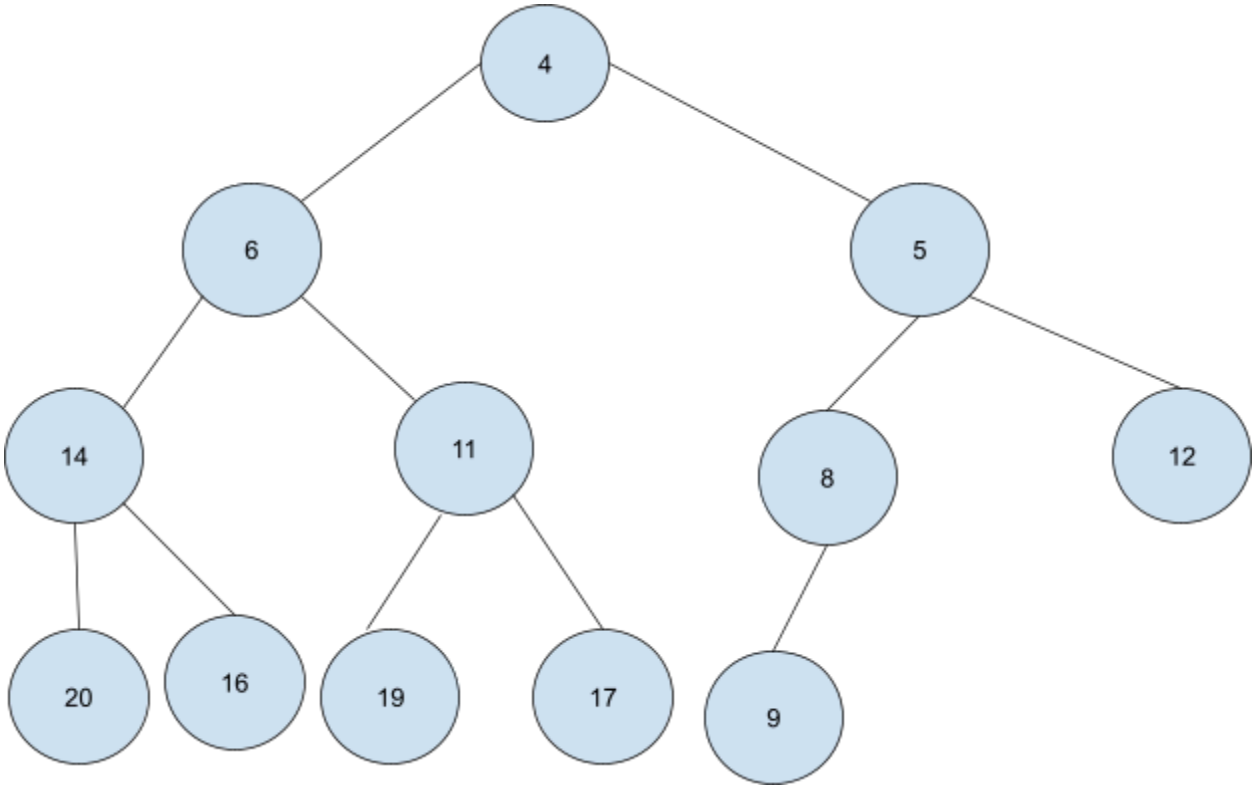
Comparing with Parent 9 and swapping



Arr=[5, 6, 8, 14, 11, 4, 12, 20, 16, 19, 17, 9]



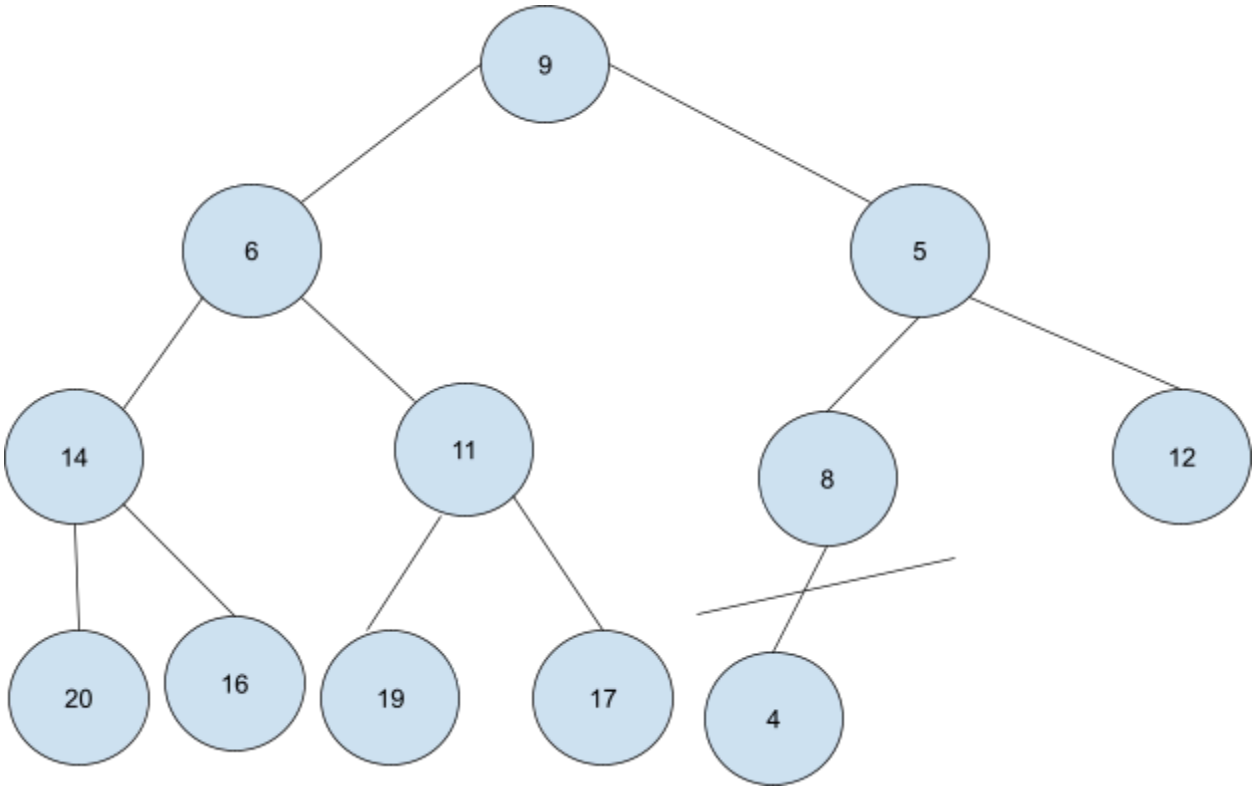
Comparing with Parent 5 and swapping then we Stop
Arr=[4, 6, 8, 14, 11, , 12, 20, 16, 19, 17, 9]



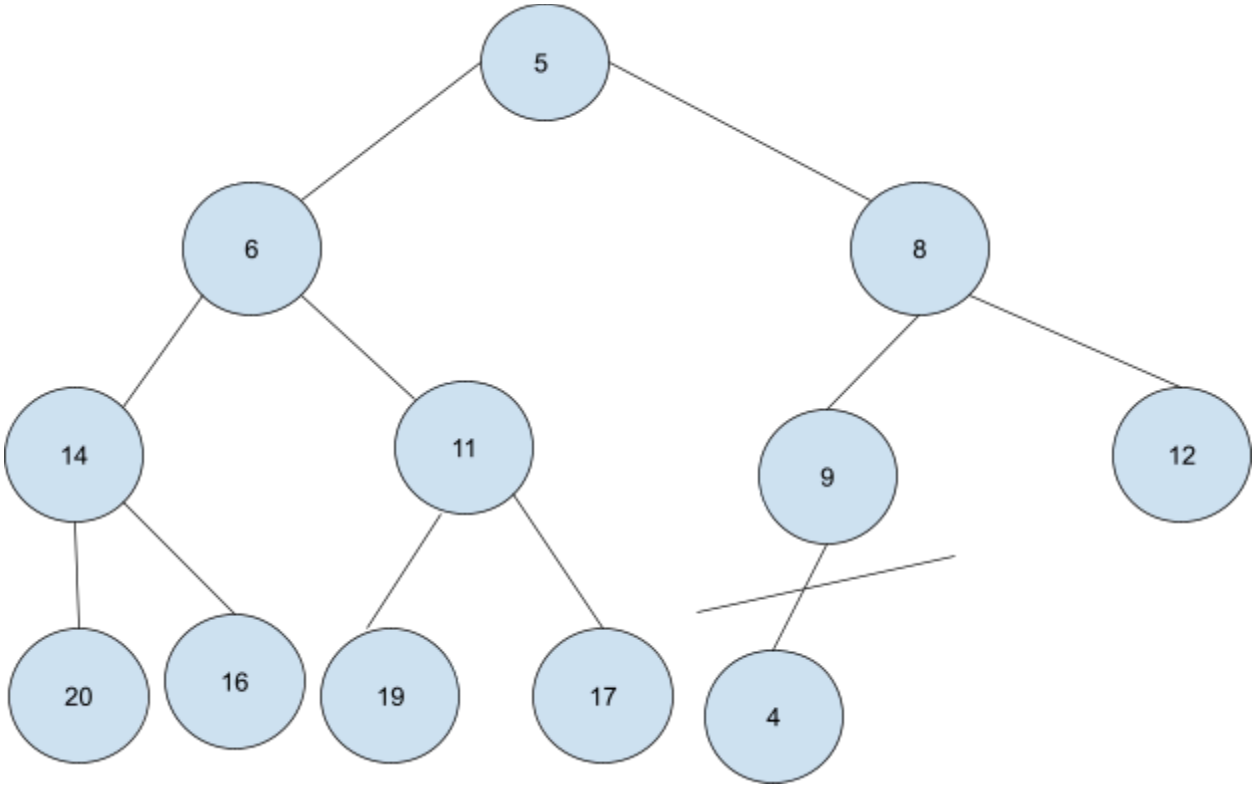
TotalSwaps=3 OverallSwaps=5

ExtractMin Operations:

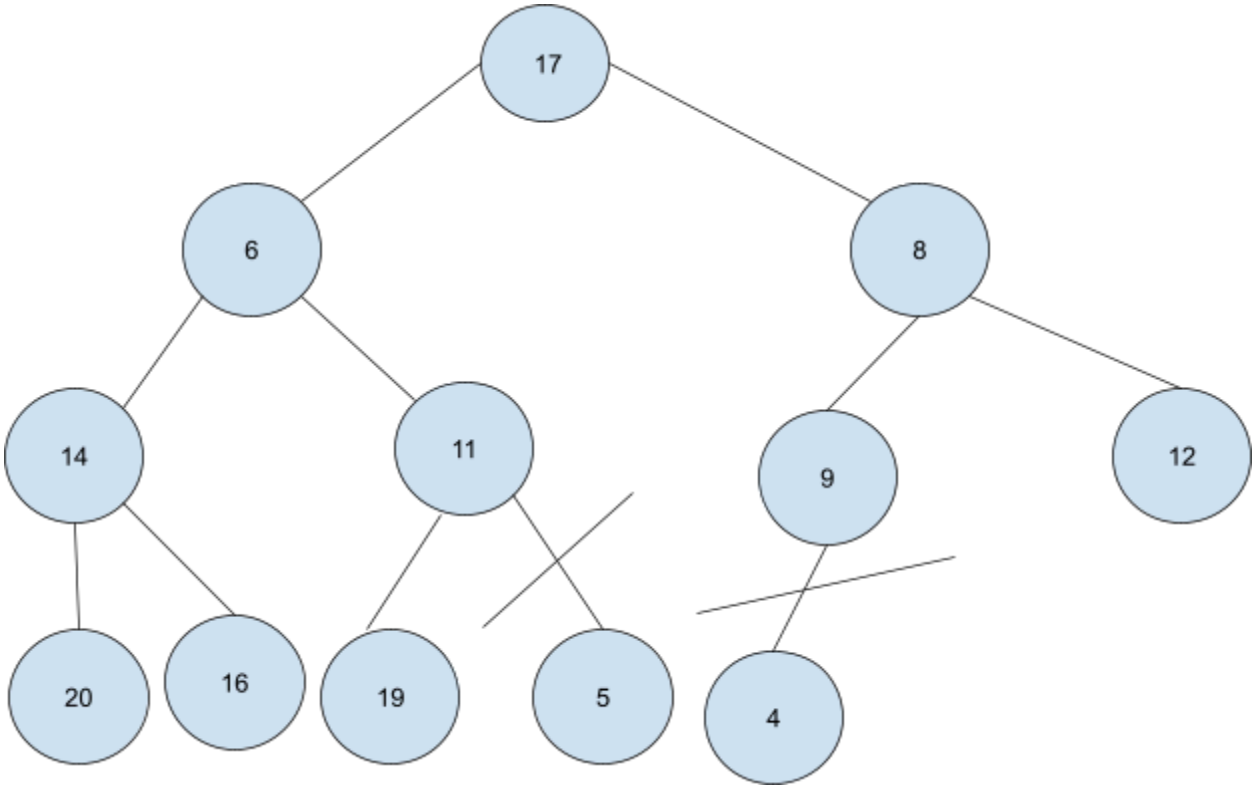
First Extract



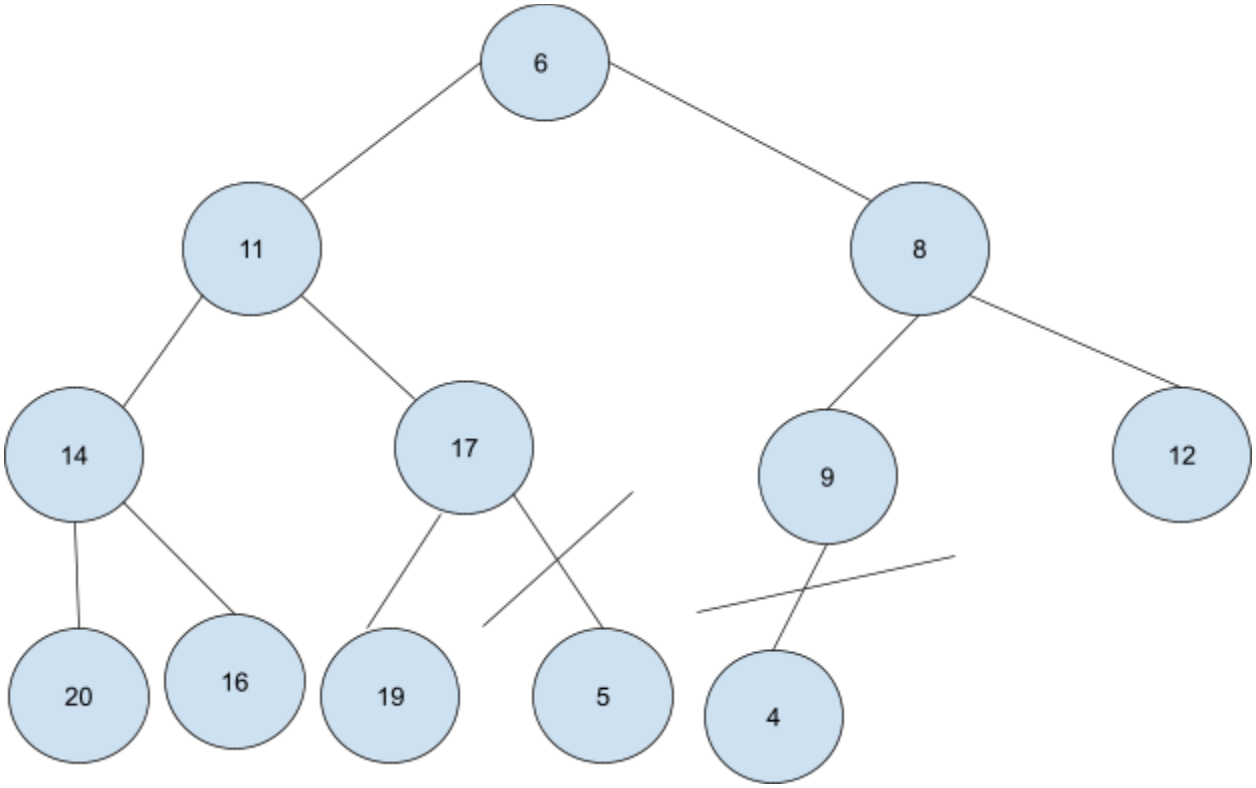
[9, 6, 5, 14, 11, 8, 12, 20, 16, 19, 17] 2 is removed
Now we will compare 9 with its children then swap if they are less than parent.
Swap 1:
[5, 6, 9, 14, 11, 8, 12, 20, 16, 19, 17]
Swap 2:
[5, 6, 8, 14, 11, 9, 12, 20, 16, 19, 17]



Second Extract:

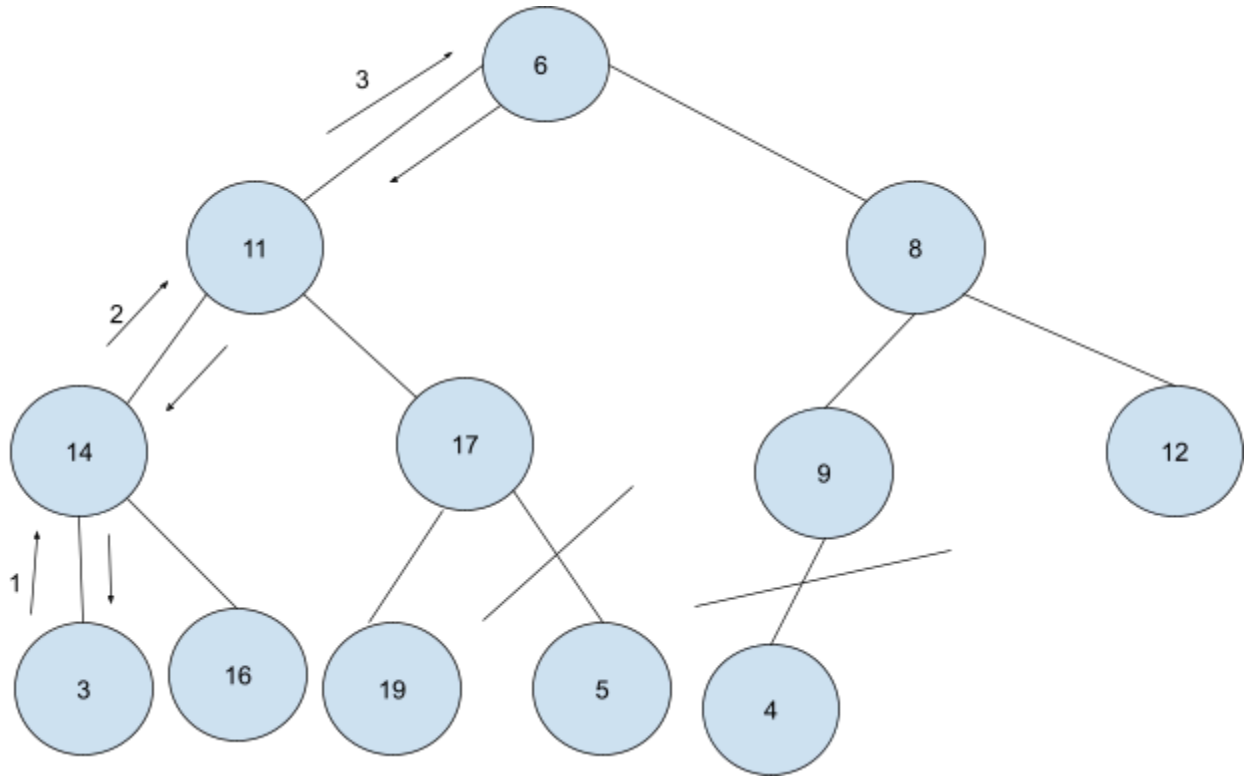


After two Swaps
[6, 11, 8, 14, 17, 9, 12, 20, 16, 19]



Decrease Key

The index 7 contains 20 so we replace it with 3 and check if the parents are greater than the child if true we swap.
[6, 11, 8, 14, 17, 9, 12, **3**, 16, 19]



Finally [3, 6, 8, 11, 17, 9, 12,14, 16, 19]
Total Swaps Required : 3
Reason:WHen a key is decreased it is necessary to check if the parents element are lesser or than key, if yes no swaps are needed, if not by the rules of heap which tells the minimum value comes before all in a binary tree we should shift the key upward

Question 5:

A)

Step	Event	Queue (Highest Priority First)	Printed
1	PUSH J1	J1(3)	—

2	PUSH J2	J2(1), J1(3)	—
3	PUSH J3	J2(1), J3(2), J1(3)	—
4	POP	J3(2), J1(3)	J2
5	PUSH J4	J4(1), J3(2), J1(3)	—
6	PUSH J5	J4(1), J3(2), J1(3), J5(3)	—
7	POP	J3(2), J1(3), J5(3)	J4
8	POP	J1(3), J5(3)	J3
9	PUSH J6	J6(2), J1(3), J5(3)	—
10	POP	J1(3), J5(3)	J6
11	POP	J5(3)	J1
12	POP	Empty	J5

B) The Tie breaking Flaw and solution

1)Push A,B,C
2)Pop:
1st output:A
2nd output:C
3rd output:B
Violating the FIFO requirement
To fix this we will store the Entries as (Priority, entry count) so when the pop is called, it will check the priority first if it is equal then it will check the entry count if it is less than the other the element B will pop, not C.
Example:
We push
A(1,1)
B(1,2)
C(1,3)
When we pop,
A will come out, afterwards B by comparing the entry count to the Entry count of C.

C) Implementing a Priority Queue

Aging is a technique used to prevent starvation by gradually increasing the priority of waiting jobs. As a job waits longer, its effective priority number decreases.

So effective Priority means the changing priority By the time.
Its formula is written as

$$P = Pi - \frac{waitingtime}{k}$$

Where P is an effective priority, Pi is original and k is the aging interval.

Code:

```
#include <iostream>
#include <queue>
#include <vector>
#include <string>
using namespace std;

struct Job {
    string name;
```

```
    int priority;
    int arrival;
};
struct Compare {
    bool operator()(Job a, Job b) {
        if (a.priority != b.priority)
            return a.priority > b.priority;

        return a.arrival > b.arrival;
    }
};

class PriorityQueue {
private:
    priority_queue<Job, vector<Job>, Compare> pq;
    int counter = 0;

public:
    void push(string job, int priority) {
        pq.push({job, priority, counter++});
    }
    string pop() {
        if (pq.empty())
            return "Queue Empty";

        string result = pq.top().name;
        pq.pop();
        return result;
    }
    string peek() {
        if (pq.empty())
            return "Queue Empty";

        return pq.top().name;
    }

    bool empty() {
        return pq.empty();
    }
};

int main() {
    PriorityQueue pq;
    pq.push("J1", 3);
```

```
    pq.push("J2", 1);
    pq.push("J3", 2);

    cout << "Step 4 POP: " << pq.pop() << endl;

    pq.push("J4", 1);
    pq.push("J5", 3);

    cout << "Step 7 POP: " << pq.pop() << endl;

    cout << "Step 8 POP: " << pq.pop() << endl;


    pq.push("J6", 2);

    for (int i = 10; i <= 12; i++) {
        cout << "Step " << i << " POP: "
             << pq.pop() << endl;
    }

    return 0;
}
```

Question 6

Merge Sort is suitable because:

- It has O(n log n) time complexity.
- It does not use Bubble, Selection, or Insertion Sort.
- It can maintain stability when students have equal CGPAs.
- It does not require any built-in sorting function.

Award Policy

The question does not specify exact CGPA eligibility thresholds, so we will use the following assumptions for this implementation:

- Rector's List: CGPA 3.90 or above.

- Dean's List: CGPA 3.50 or above but below 3.90.
- No Award: CGPA below 3.50.

Code in Cpp file.

Question 7

```
#include <iostream>
#include <string>
using namespace std;

struct Parcel {
    int trackingNumber;
    string sender;
    string destination;
    int regionCode;
};

int getMax(Parcel arr[], int n) {
    int maximum = arr[0].trackingNumber;

    for (int i = 1; i < n; i++) {
        if (arr[i].trackingNumber > maximum)
            maximum = arr[i].trackingNumber;
    }

    return maximum;
}

void countingSort(Parcel arr[], int n, int exp) {

    Parcel* output = new Parcel[n];
    int count[10] = {0};

    for (int i = 0; i < n; i++) {
        int digit = (arr[i].trackingNumber / exp) % 10;
        count[digit]++;
    }

    for (int i = 1; i < 10; i++)
        count[i] += count[i - 1];

    for (int i = n - 1; i >= 0; i--) {
```

```
        int digit = (arr[i].trackingNumber / exp) % 10;

        output[count[digit] - 1] = arr[i];
        count[digit]--;
    }

    for (int i = 0; i < n; i++)
        arr[i] = output[i];

    delete[] output;
}

void radixSort(Parcel arr[], int n) {

    int maximum = getMax(arr, n);

    for (int exp = 1; maximum / exp > 0; exp *= 10) {
        countingSort(arr, n, exp);
    }
}
//Part B
struct Node {
    Parcel data;
    Node* next;

    Node(Parcel p) {
        data = p;
        next = nullptr;
    }
};

Node* getMiddle(Node* head) {

    if (head == nullptr)
        return head;

    Node* slow = head;
    Node* fast = head->next;

    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next;
        fast = fast->next->next;
    }
```

```
    return slow;
}
Node* mergeLists(Node* a, Node* b) {

    if (a == nullptr)
        return b;

    if (b == nullptr)
        return a;

    Node* result;

    if (a->data.regionCode <= b->data.regionCode) {

        result = a;
        result->next = mergeLists(a->next, b);
    }
    else {

        result = b;
        result->next = mergeLists(a, b->next);
    }

    return result;
}

Node* mergeSort(Node* head) {

    if (head == nullptr || head->next == nullptr)
        return head;

    Node* middle = getMiddle(head);

    Node* secondHalf = middle->next;
    middle->next = nullptr;

    Node* left = mergeSort(head);
    Node* right = mergeSort(secondHalf);

    return mergeLists(left, right);
}

void insertEnd(Node*& head, Parcel p) {
```

```
    Node* newNode = new Node(p);

    if (head == nullptr) {
        head = newNode;
        return;
    }

    Node* temp = head;

    while (temp->next != nullptr)
        temp = temp->next;

    temp->next = newNode;
}

void displayList(Node* head) {

    while (head != nullptr) {

        cout << head->data.trackingNumber
             << " | "
             << head->data.sender
             << " | "
             << head->data.destination
             << " | Region: "
             << head->data.regionCode
             << endl;

        head = head->next;
    }
}

int main() {

    Node* head = nullptr;

    insertEnd(head, {1005, "Ali", "Karachi", 3});
    insertEnd(head, {1002, "Ahmed", "Lahore", 1});
    insertEnd(head, {1008, "Sara", "Islamabad", 2});
    insertEnd(head, {1009, "Hassan", "Multan", 1});
    insertEnd(head, {1010, "Zara", "Karachi", 3});

    head = mergeSort(head);
```

```
    cout << "Parcels Sorted By Region:\n";

    displayList(head);

    return 0;
}
```

Question 8:

```
#include <iostream>
using namespace std;

#define N 4

int maze[N][N] = {
    {1, 1, 0, 1},
    {1, 1, 1, 1},
    {0, 1, 0, 1},
    {1, 1, 1, 1}
};
bool visited[N][N] = {false};
char path[100];
int pathIndex = 0;
bool found = false;

bool isSafe(int row, int col) {
    if (row < 0 || row >= N ||
        col < 0 || col >= N) {
        return false;
    }

    if (maze[row][col] == 0 || visited[row][col]) {
        return false;
    }

    return true;
}
void findPaths(int row, int col) {
    if (row == N - 1 && col == N - 1) {
        found = true;

        for (int i = 0; i < pathIndex; i++) {
```

```
            cout << path[i];
        }

        cout << endl;
        return;
    }
    visited[row][col] = true;
    if (isSafe(row - 1, col)) {
        path[pathIndex++] = 'U';
        findPaths(row - 1, col);
        pathIndex--;
    }
    if (isSafe(row + 1, col)) {
        path[pathIndex++] = 'D';
        findPaths(row + 1, col);
        pathIndex--;
    }
    if (isSafe(row, col - 1)) {
        path[pathIndex++] = 'L';
        findPaths(row, col - 1);
        pathIndex--;
    }

    if (isSafe(row, col + 1)) {
        path[pathIndex++] = 'R';
        findPaths(row, col + 1);
        pathIndex--;
    }

    visited[row][col] = false;
}

int main() {

    if (maze[0][0] == 0 || maze[N-1][N-1] == 0) {
        cout << "No path exists." << endl;
        return 0;
    }

    findPaths(0, 0);

    if (!found) {
        cout << "No path exists." << endl;
    }
```

```
    return 0;
}
```

Question 9:

```
#include <iostream>
using namespace std;

#define ROWS 6
#define COLS 8
#define COURSES 4

string course[COURSES] = {
    "Programming",
    "Database Systems",
    "Operating Systems",
    "Computer Networks"
};

int seat[COURSES];
bool colUsed[COLS];

int count = 0;
bool isSafe(int row, int col, int courseNo) {
    if (row == 3 && col == 3) {
        return false;
    }
    for (int i = 0; i < courseNo; i++) {

        int prevRow = i;
        int prevCol = seat[i];

        if (prevCol == col) {
            return false;
        }
        if (abs(prevRow - row) == abs(prevCol - col)) {
            return false;
        }
    }

    return true;
```

```
}
void assignSeats(int courseNo) {

    if (courseNo == COURSES) {

        count++;

        cout << "\nArrangement " << count << ":\n";

        for (int i = 0; i < COURSES; i++) {
            cout << course[i] << " -> Row "
                 << i << ", Column " << seat[i] << endl;
        }

        return;
    }
    if (courseNo == 0) {

        int row = 0;

        for (int col = 0; col < COLS; col++) {

            if (isSafe(row, col, courseNo)) {

                seat[courseNo] = col;
                colUsed[col] = true;

                assignSeats(courseNo + 1);
                colUsed[col] = false;
            }
        }
    }

    else {

        int row = courseNo;

        for (int col = 0; col < COLS; col++) {

            if (isSafe(row, col, courseNo)) {

                seat[courseNo] = col;
                colUsed[col] = true;
```

```
            assignSeats(courseNo + 1);
            colUsed[col] = false;
        }
    }
  }
}

int main() {

  assignSeats(0);

  if (count == 0) {
    cout << "\nNo valid seating arrangement exists.\n";
  }
  else {
    cout << "\nTotal valid arrangements: " << count << endl;
  }

  return 0;
}
```

Question 10:

```
#include <iostream>
using namespace std;

int digits[] = {1, 2, 3, 4, 5};
int totalDigits = 5;

int N = 3;
int S = 9;

int combination[100];
int count = 0;
void generate(int start, int index, int sum) {
  if (index == N) {

    if (sum == S) {
      count++;

      for (int i = 0; i < N; i++) {
        cout << combination[i] << " ";
```

```
            }

            cout << endl;
        }

        return;
    }
    for (int i = start; i < totalDigits; i++) {

        combination[index] = digits[i];

        generate(i + 1, index + 1, sum + digits[i]);

        combination[index] = 0;
    }
}

int main() {

    generate(0, 0, 0);

    if (count == 0) {
        cout << "No valid combination exists." << endl;
    }
    else {
        cout << "\nTotal valid combinations: " << count << endl;
    }

    return 0;
}
```